

Agent on a Leash

Technical Report

The data, the problem, and every component we built to solve it — with the reasoning behind each decision.

PURCHASES, LIVE	SLOWEST DECISION	BUDGET	TESTS
45	2.88 _{ms}	8000 _{ms}	301

The brief, and where we stand against it

Viseca published a 25-item solution checklist alongside the challenge. Every item is scored here honestly, because a claim we cannot defend is worse than a gap we admit.

Core components

REQUIREMENT	WHERE IT LIVES	STATE
Policy interpretation engine	<code>policy/compiler.py</code> — instruction to executable rules	DONE
Transaction evaluation logic	<code>decision/rules.py</code> + <code>decision/facts.py</code>	DONE
Decision engine (approve / decline / ask)	<code>decision/engine.py</code> , seven stages	DONE
Human approval / rejection path	Step-up inbox in the UI, <code>/resolve</code> on the platform	DONE
Policy update / revocation	<code>policy/amend.py</code> , tighten-only; revoke in one click	DONE

Testing and validation

REQUIREMENT	EVIDENCE	STATE
Ordinary transaction	AU0035 approved in 0.39 ms with no adverse findings	DONE
Ambiguous / suspicious transaction	AU0040 injection, AU0039 lookalike, AU0016 unstated terms	DONE
Human escalation path	5 purchases escalated live; both resolve paths exercised	DONE
Works with synthetic data	All 45 offline, no network, no key	DONE
Connects to live API	All 45 answered on the sandbox, zero parse failures	DONE

Quality and resilience

REQUIREMENT	EVIDENCE	STATE
Meets latency requirements	2.88 ms worst case against 8000 ms	DONE
Works if external services fail	Engine has no network. Model failure tested across six modes	DONE
No hard-coded scenario assumptions	A test rewrites scenario ID, request ID and replay order; decisions unchanged	DONE
Decisions explainable and auditable	Every verdict carries its findings; append-only journal	DONE
Customer retains control	Draft, confirm, tighten, revoke; nothing runs before confirmation	DONE

Demonstration and documentation

REQUIREMENT	EVIDENCE	STATE
All three demo flows	<code>leash demo</code> selects them from a live run, not a script	DONE
Judges understand what / why / how	Evidence panel per decision, in the customer's own terms	DONE
Architecture diagram	<code>ARCHITECTURE.md</code> , and section 04 of this report	DONE
Policy format documented	19-field closed namespace, section 05	DONE
API usage examples, deployment, demo script	<code>README.md</code> , <code>submission/recording-guide.pdf</code>	DONE

One item we should not overclaim.

The brief suggests combining “rules-based logic, behavioural signals, machine learning, language models and interface design”. We use four of those five. There is **no trained machine-learning model** in this system: behavioural signals are computed arithmetically from history rather than learned. We think that is the right call at this data scale — 4,701 history rows is not a training set — but it is a deliberate omission, not an oversight, and we would rather say so than imply otherwise.

What we were given, and what is hidden in it

Eleven CSVs, three JSON schemas and a manifest. The structure is a realistic payment hierarchy, and several of its properties are traps rather than conveniences.

Customer —< Accounts —< Cards —< Authorizations

FILE	ROWS	WHAT IT CARRIES, AND WHY IT MATTERS
<code>purchase_attempts</code>	45	The purchases to decide. Ordered by <code>replay_order</code> within each of 5 scenarios.
<code>purchase_attempt_items</code>	56	Cart lines. <code>item_details</code> is merchant-written free text — the attack surface.
<code>authorization_history</code>	4,701	Past activity. The only source for familiarity, device history and velocity. 453 are agent purchases (416 approved, 37 declined) — deliberately included so agent use is not itself treated as anomalous.
<code>merchants</code>	58	Name, category, MCC, country, availability. Contains the lookalike pair.
<code>items</code>	66	Catalogue with CHF price ranges. Three item categories (<code>cosmetics</code> , <code>gift_card</code> , <code>membership</code>) have no merchant counterpart at all.
<code>customers</code> / <code>accounts</code> / <code>cards</code>	92	Personas, issuer limits, card capabilities and status.
<code>scenario_catalogue</code>	5	The five cardholder instructions, verbatim.
<code>scenario_authorities</code>	5	Which customer and card replays each scenario. Not a wallet policy.
<code>fx_rates</code>	4	Fixed synthetic rates: EUR 0.95, GBP 1.12, USD 0.87, CHF 1.00.

Six properties of the data that change the design

PROPERTY	WHAT IT FORCES
No expected decisions, no risk labels, no answer key	Nothing can be validated against ground truth. Correctness has to be argued from policy and facts, which is why every verdict carries its evidence.
Currency does not follow merchant country. 61 historical rows at foreign merchants bill in CHF.	Convert using the row's <code>currency</code> , never the merchant's country.
Historical and runtime authorizations share a column name but live in different ID namespaces (<code>TR...</code> vs <code>AU...</code>)	A join across the two files returns nothing, correctly. Never derive one ID from another.
<code>card_present</code> is not a restatement of <code>channel</code> . <code>mobile_wallet</code> is genuinely mixed: 415 in-person, 197 remote.	Treating the wallet channel as uniformly remote misreads a third of it.
Merchant <code>availability</code> is catalogue metadata, not a validation rule	A shop with a physical store still appears in ecommerce records.
Three item categories have no merchant counterpart	A supermarket sells cosmetics without ceasing to be a grocery merchant — so a purpose rule must constrain <code>items[]</code> , not <code>merchant_category</code> .

The five scenarios, and what each is really testing

ID	EVENTS	THE TRAP
<code>SCEN0000</code>	1	Connection check. Does the end-to-end path work at all.
<code>SCEN0001</code>	10	Per-order and rolling seven-day limits. Delivery fee already inside the total. Two orders six minutes apart. A cosmetics line inside a grocery basket.
<code>SCEN0002</code>	12	Size 42 instead of 43. Final sale. A 7-day return window against a 14-day requirement. An unstated window. A <i>trail</i> shoe where a <i>road</i> shoe was asked for. An unrequested protection plan. A cycling helmet. CHF 215 against a CHF 200 cap. A shop outside the stated category. And one unfamiliar but fully compliant seller, which must <i>not</i> be blocked.
<code>SCEN0003</code>	11	A new device appearing at 02:14. A velocity burst of 1 → 2 → 3 attempts in ten minutes. Foreign merchants in IT and GB requiring FX. Then recovery to the familiar device, which must relax again.
<code>SCEN0004</code>	11	Two prompt injections. A lookalike seller one letter from a familiar one. A duplicate basket under a new ID. A gift voucher instead of a monitor. An unrequested add-on. And a legitimate re-quote after a decline, which must be allowed.

Four safety concerns, one decision

The brief names four things that can go wrong when an agent holds a card. They fail in different directions, and a system that only guards against one of them is not a control layer.

CONCERN	HOW IT ACTUALLY MANIFESTS IN THE DATA	OUR ANSWER
Spending limits “Doesn't spend too much?”	Not by ignoring a limit, but by splitting an order, by the delivery fee, or by a rolling window nobody is tracking.	Decimal arithmetic, rolling windows on simulated purchase time, only final approvals counted.
Merchant vetting “Doesn't buy from shady merchants?”	The shop is not obviously shady — its name is one letter from one you trust.	Familiarity matched on <code>merchant_id</code> , plus similarity detection against names this card has actually used.
Intent verification “Purchases what you actually intended?”	The right category, the right price, the wrong item. A trail shoe. Size 42. A voucher.	Requested-item matching on name and category, attribute extraction from product text, unrequested-line detection.
Prompt injection “Resilient against injection?”	The shop writes the product description, and that description is an input to whatever reads it.	Facts extracted by pattern only; the scanner holds no reference to the policy, so nothing written can reach the verdict.

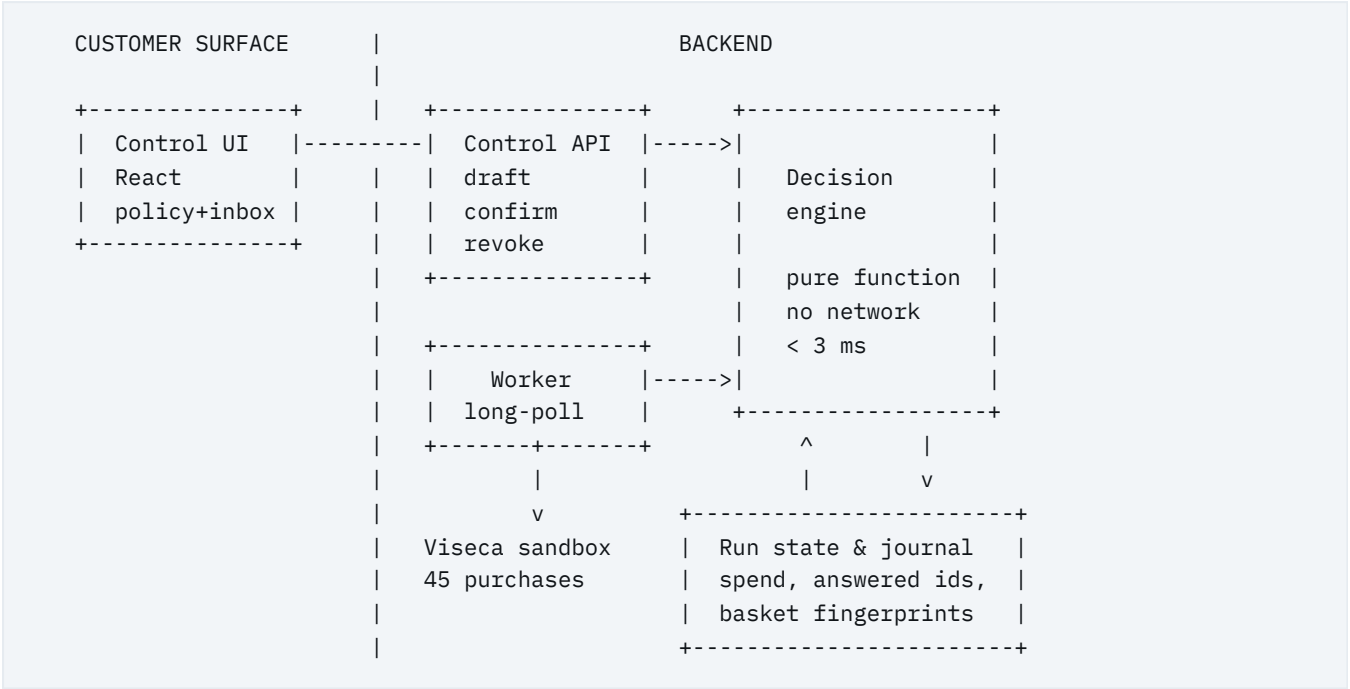
And the failure mode nobody lists

Blocking ordinary shopping is also a failure.

The brief says it explicitly: “Unfamiliar purchases are not automatically wrong; blocking ordinary shopping unnecessarily is also a failure.” A layer that declines everything scores zero on both counts. Two of the supplied attempts exist purely to catch over-blocking — AU0023, a new but fully compliant seller, and AU0042, a legitimate re-quote after a decline. Both are approved.

Two deployables, one contract

Viseca intends to fold wallet control into the existing *one* app, while the decision runs in the backend under a hard latency ceiling. Opposite requirements, so they never share a process.



The dependency graph points one way only, and this was verified by grepping imports rather than assumed: `models/`, `policy/` and `decision/` never import `api/` or `worker/`, and contain no HTTP client at all.

The engine is a pure function of (event, run state).

No network, no clock beyond what it is handed, no I/O. One constraint buys three properties: it is fast enough for the deadline, it is testable offline against every supplied purchase, and it cannot be talked out of an answer because there is nothing to talk to.

MODULE	LINES	RESPONSIBILITY
<code>models/</code>	~700	Typed event contract, Decimal money, 13 closed vocabularies
<code>policy/</code>	~800	Instruction to rules; 19-field namespace; tighten-only amendments
<code>decision/</code>	~1,270	Seven stages, fact resolution, sanitiser, run state, card history
<code>replay/</code>	~600	Offline event assembly from CSV, harness, append-only journal
<code>api/</code>	~800	Sandbox client (15 endpoints), control plane, session persistence
<code>worker/</code>	~400	Long-poll loop, idempotency, step-up inbox, one-command live run
<code>llm/</code>	~600	Optional advisor and clarifier across three providers
<code>ui/</code>	~900	Policy composer, decision timeline, step-up inbox

Twelve components, and the reasoning behind each

1. Data pack and integrity

datapack.py · 4 tests

The pack is vendored so offline replay needs no network. Every one of its 18 files is verified against the manifest's sha256 and row count on each run.

Why: a silently edited fixture would change decisions without changing code. Failing loudly at load is cheaper than debugging a wrong verdict.

2. The typed contract

models/ · 16 tests

Two gates on every incoming event: the platform's own `authorization_event.schema.json`, then Pydantic in **strict** mode with `extra="forbid"`. All problems are reported at once, never a partial parse.

Strict mode gives the schema's semantics rather than a friendly approximation. `"20.00"` is rejected where a number is required. `5411` is rejected for `merchant_mcc`, which the schema pins as a four-digit *string*. `True` is rejected wherever an integer is required, because Python treats `bool` as `int` and the schema does not.

Three semantics the schema cannot express, added on top: `order_returnable` stays a four-valued string, so “unknown” never becomes “yes”. A `null` period spend stays null, never 0.0, which would invent headroom. Naive timestamps are rejected, because a spending window computed against a zoneless time is a quietly wrong window.

3. The policy compiler

policy/compiler.py · 39 tests

Turns the customer's own sentence into three outputs. `hard_rules` are executable. `guidance` is what we understood, in plain language. `open_questions` is what the sentence does not settle — and we refuse to settle it for them.

Extraction is deterministic pattern matching, no model. Compilation happens once, before confirmation, and the same sentence must always produce the same rules, because a customer confirming a policy is agreeing to *those* rules.

Example – SCEN0002 compiles to 7 rules: `billing_amount_chf <= 200`, `merchant_category in [sporting_goods]`, `return_window_days >= 14`, `order_returnable = true`, `requested_item = "road-running shoes"`, `requested_attribute = size=43`, `requested_item_match = match` – plus 4 open questions.

What it refuses to decide. “A shop I use regularly” has no field in the event. Somebody must decide whether *regularly* means one past purchase or five, and that somebody is the customer. We also surface a gap the instructions never mention but the data does: two orders minutes apart can each sit under a per-order cap while together clearing it.

4. The field namespace

policy/vocabulary.py · 19 fields

A rule's `field` is, in the contract's words, “a convention for your engine to interpret”. That freedom is also a hazard: a field name nothing resolves is a rule that silently never fires. So the namespace is closed, declared, and every compiled rule is checked against it.

Each field declares how it resolves — straight off the event, across *every* cart line, or derived from history, run state or merchant text — and whether a breach declines or escalates.

Only one field escalates on breach: `derived.session_integrity`. Whether someone other than the cardholder is driving a session is an inference, never something the evidence settles. An unfamiliar device is a new laptop as often as an intruder, and customers phrase that constraint as “pause”.

5. The decision engine

decision/engine.py · 34 tests

Seven stages in order, each appending to a shared evidence ledger. The verdict is derived from the ledger at the end rather than returned early from the middle, so the explanation always accounts for everything looked at.

#	STAGE	WHAT IT PREVENTS
1	Parse and validate	Deciding on a message we could not fully read
2	Preconditions	Spending on a permission already withdrawn
3	Sanitise merchant text	A product description raising a spending limit
4	Hard rules	A period limit measured on prior spend alone
5	Intent match	A trail shoe passing as a road shoe
6	Behavioural signals	Matching familiarity on a name an impostor controls
7	Resolve	Collapsing “uncertain” into either neighbour

Resolution rule: a definite breach declines; anything unsettled falls to the customer's own `uncertainty_policy`; only a clean pass approves.

6. The sanitiser

decision/sanitize.py · 13 tests

`item_details` is genuinely useful — shoe size, panel size, return window — and it is written by whoever wants the payment approved. Those two facts are only in tension if the text is obeyed.

So the module does two separate things and keeps them separate. Facts are extracted by pattern, and **a pattern can only ever yield a number or a token**, so no phrasing produces a permission. Separately, text addressed to an automated decider is detected across six independent signatures, after folding zero-width joiners, bidirectional overrides and full-width characters that would otherwise hide it.

Signatures: `addresses_automated_agent · impersonates_system_role · overrides_prior_instructions · claims_pre_authorisation · demands_immediate_approval · raises_or_removes_a_limit`

False positives matter as much. A test asserts that exactly 2 of 56 supplied cart lines flag. Ordinary copy — “Road-running shoe, size 43; returns accepted within 30 days” — passes untouched.

7. Money and time

models/money.py · 6 tests

Spending limits are the spine of the challenge, so money never touches binary floating point. Amounts are `Decimal`, quantised to two places with half-even rounding, converted at the pack's fixed rates read from `fx_rates.csv` rather than hard-coded.

Two clocks are kept apart. `authorization.timestamp` is simulated purchase time and drives every spending window, velocity count and familiarity calculation. `deadline_at` is the real clock and governs only the response.

Both money invariants are verified against all 45 attempts as tests: our CHF conversion reproduces the platform's `billing_amount_chf` exactly (219.00 GBP → 245.28, 450.00 USD → 391.50), and `items_subtotal + delivery_fee` equals `amount` on every row – so adding delivery again would double-charge.

8. Run memory

decision/state.py · 10 tests

Three things survive between purchases, each mapping to a documented failure mode. **Approved spend**, counting only final approvals – a purchase awaiting a human is not yet spend, and is tracked separately. **Answers already given**, keyed by live `authorization_id`, so a redelivery replays the saved answer rather than charging the limit twice. **Basket fingerprints**, because “different IDs can still describe an unwanted duplicate order”.

The platform redelivers after 3 seconds, which we learned from `/v1/bootstrap` rather than from the written contract.

9. Offline replay and the journal

replay/ · 10 tests

The pack ships a *readable* copy of one attempt, not a live event – it has string amounts and a flat shape, and it does not validate against the schema. So offline events are assembled from the CSVs instead, following the documented conversion rules, with fresh real-clock deadlines and live IDs minted per run.

Every decision is journalled append-only with its full evidence. A worker that restarts rebuilds spend totals and answered IDs from it, because coming back with an empty total would hand the agent its budget a second time.

10. Sandbox client and worker

api/ + worker/ · 55 tests

All 15 endpoints. The worker answers what it is handed and never waits on a person. A `204` means no work in that polling window, *not* that the run ended. A `step_up` queues the purchase and the reply goes back through `/resolve`, never as a second automated decision. An event failing the contract is escalated with the parse errors attached rather than decided on.

`leash run` does the whole live sequence in one command, with two orderings enforced by tests: the worker exists before the run starts, because the eight-second deadline runs from *queueing*; and nothing runs before the policy is confirmed.

11. Control plane and interface

api/control.py + ui/ · 24 tests

A separate FastAPI service and a React interface that talks only to it. The engine never imports from the control plane; the control plane calls into the engine.

Built around the order of authority rather than around screens: an instruction is compiled and shown back with its rules, its plain-language reading and its open questions; nothing runs before confirmation; a live policy can be tightened and never loosened; a paused purchase sits in an inbox that says plainly it is not approved and does not count against a limit.

The session — mandate, inbox, runs and journal — is written atomically to disk on every change and restored on startup. Anything unreadable is discarded rather than half-restored.

12. The optional model layer

llm/ · 25 tests

Three providers, differing only in endpoint and auth. Two roles: an **intent advisor** that gives a second opinion on whether a cart line is the item described, and a **policy clarifier** that rewords a blocking compiler question and rewrites the instruction from the customer's answer.

Both are constrained the same way. The advisor never sees the policy and is one-directional — it can turn an apparent match into uncertainty, never the reverse. The clarifier changes wording only; code decides *when* to ask and which answers are acceptable.

Every failure path returns nothing: no key, no network, a timeout, prose instead of JSON, a verdict outside the three accepted, a reply arriving after the budget. One budget covers a whole event, not one per cart line.

All 45 purchases on the live API

SCENARIO	APPROVE	DECLINE	ASK	SLOWEST
SCEN0000 Connection check	1	0	0	0.32 ms
SCEN0001 Household budget	5	5	0	0.58 ms
SCEN0002 Item and terms	3	7	2	1.48 ms
SCEN0003 Session integrity	4	6	1	2.88 ms
SCEN0004 Manipulated agent	4	5	2	2.57 ms
Total	17	23	5	2.88 ms

Zero parse failures, zero missed deadlines. The counts match the offline replay exactly, which is the strongest available evidence that the engine is deterministic and that the offline harness is faithful.

Eight defects found, and how

A period limit excluded the order being decided

CHF 299.50 + 65.50 passed a CHF 300 cap; the next order was blamed. Found by running SCEN0001.

An unstated return policy was declining

Not permission — but not a refusal either. Found on AU0016.

A trail shoe passed as a road-running shoe

Its copy advertises a “lugged off-road sole”; the matcher read prose as identity. Found on AU0017.

A session signal declined instead of pausing

The customer wrote “pause”. Found on AU0026.

Our own serialiser produced invalid events

Decimal written as a JSON string. Found by a round-trip test.

The live envelope did not parse

`event_id` arrives as an integer where the contract gives it no type. Found on first contact with the real API.

“CHF 300 a week” compiled to a per-order cap

A weekly budget became a per-order allowance spendable daily. Found while driving the clarification flow.

Answering a paused purchase put it straight back

The interface re-posted the run instead of reading it, replaying every purchase. Found by using the interface.

What judges are likely to ask

“Where is the AI?”

The AI is the thing being controlled, not the thing controlling. We built the model hook, measured it, and found the deterministic engine did not need it: slowest decision went from 0.97 ms to 3,102 ms — 39% of the platform's budget — and not one decision changed. Putting a model in the path that gates spending would be the vulnerability, in a challenge whose whole point is text trying to talk a decider into approving.

“How do you know it is not hard-coded to the scenarios?”

A test rewrites `scenario_id`, `request_id` and `replay_order` on every event and asserts the decisions are unchanged. Nothing in the engine reads any of the three.

“Why did it decline 23 of 45? Isn't that over-blocking?”

Every decline traces to a stated customer rule, and the evidence panel names which. Two attempts exist specifically to catch over-blocking — a new but compliant seller and a legitimate re-quote after a decline — and both are approved. SCEN0001 declines five in a row only because the customer's own CHF 300 weekly limit was reached.

“What happens if the customer never answers?”

Nothing is sent. We measured the live API: an unanswered purchase moves to `timed_out` on its own at exactly 120 seconds. A decline we submitted would be recorded as the customer's decision when they never made one — which the guide explicitly forbids. Lapsing is the absence of an answer, not an answer.

“Could a merchant get a purchase approved by writing the right text?”

No, and not because we filter well. The sanitiser returns data and holds no reference to the policy, and facts are extracted by patterns that can only yield a number or a token. There is no code path by which merchant text reaches the verdict.

“What is the weakest part?”

The instruction compiler is regex-based, so it handles phrasings we anticipated and misses others. The mitigation is that everything it extracts is shown back before confirmation, so a bad reading is visible rather than silent. Two tuned numbers — familiarity at one prior purchase, a three-hour duplicate window — are judgements, not measurements.

“Would this scale?”

The engine is a pure function with no I/O, so it scales horizontally without coordination. Two things would not: familiarity currently loads all 4,701 history rows at import and should be a query, and the control session is a JSON file rather than a database.

The three-minute path

MIN	SHOW	REQUIREMENT
0:00	Compile a policy, point at the questions we refuse to answer	Customer control
0:50	Authorise, run SCEN0004, point at AU0035 approved in 0.39 ms	1. Ordinary, no friction
1:30	Expand AU0040's findings — inside every limit, stopped by its own description	2. Useful intervention
2:20	Answer one paused purchase, let the second lapse, then revoke	3. Human path
3:00	<code>leash run --scenario SCEN0004</code> against the live sandbox	It is real